An Introduction of
Support Vector Machine

Slides are from Dr. Belhumuer, Columbia University

Support Vector Machine (SVM)

- A classifier derived from statistical learning theory by Vapnik, et al. in 1992
- SVM became famous when, using images as input, it gave accuracy comparable to neural-network with hand-designed features in a handwriting recognition task
- SVM has been widely used in object detection & recognition, content-based image retrieval, text recognition, biometrics, speech recognition, etc.
- Also used for regression

- Chapter 5.1, 5.2, 5.3, 5.11 (5.4*) in textbook



V. Vapnik

Outline

- Linear Discriminant Function
- Large Margin Linear Classifier
- Nonlinear SVM: The Kernel Trick

Discriminant Function

- Chapter 2.4: the classifier is said to assign a feature vector x to class w_i if

$$f_i(\mathbf{x}) > f_j(\mathbf{x}) \qquad \text{for all } j \neq i$$

- For two-category case, $F(\mathbf{x}) \equiv f_1(\mathbf{x}) - f_2(\mathbf{x})$

Decide ω_1 if $f(\mathbf{x}) > 0$; otherwise decide ω_2

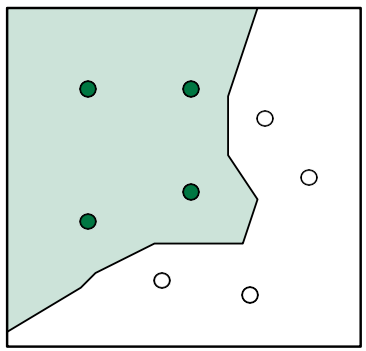
- An example we've learned before:

Minimum-Error-Rate Classifier

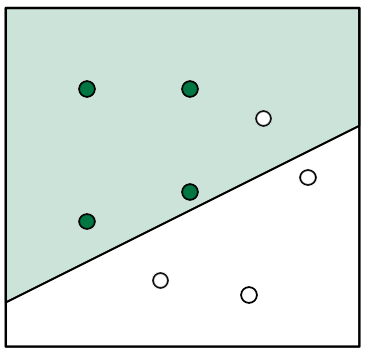
$$f(\mathbf{x}) \equiv p(\omega_1 \mid \mathbf{x}) - p(\omega_2 \mid \mathbf{x})$$

Discriminant Function

- It can be arbitrary functions of *x*, such as:

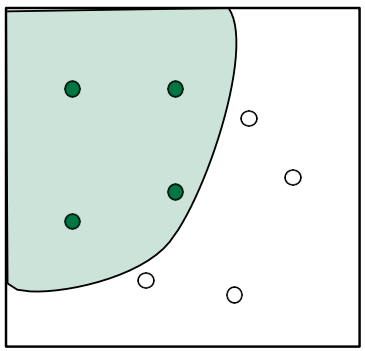


Nearest Neighbor



Linear Functions

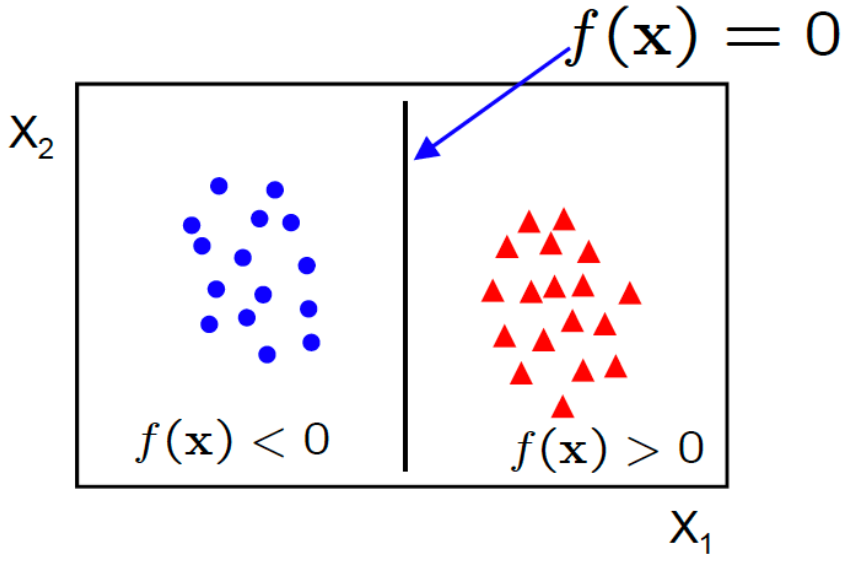
$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$



Nonlinear Functions

A linear classifier has the form

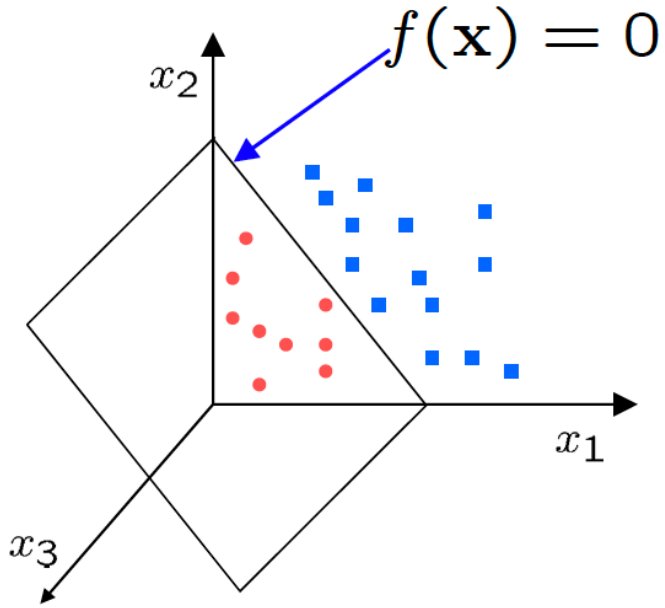
$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$



- in 2D the discriminant is a line
- $\mathbf{w}$ is the normal to the line, and b the bias
- $\mathbf{w}$ is known as the weight vector

A linear classifier has the form

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$



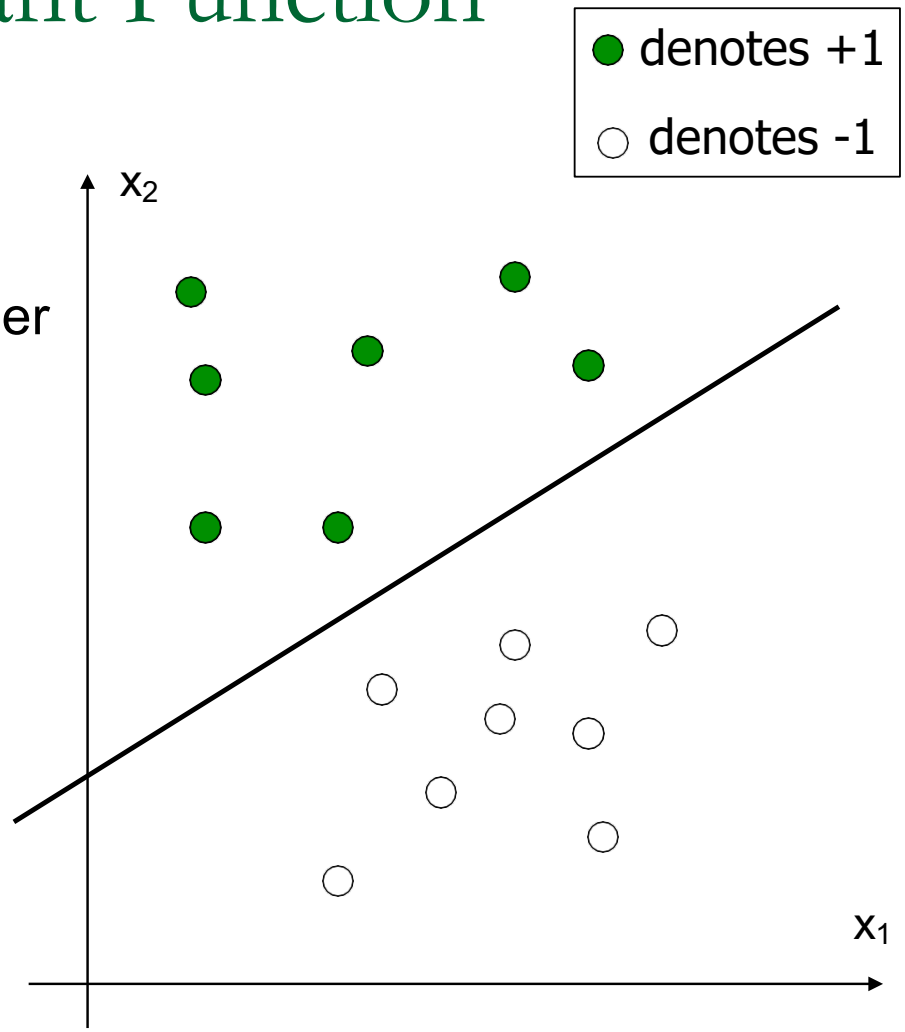
- in 3D the discriminant is a plane, and in nD it is a hyperplane

- For a K-NN classifier it was necessary to `carry' the training data
- For a linear classifier, the training data is used to learn w and then discarded
- Only w is needed for classifying new data

Linear Discriminant Function

- How would you classify these points using a linear discriminant function in order to minimize the error rate?

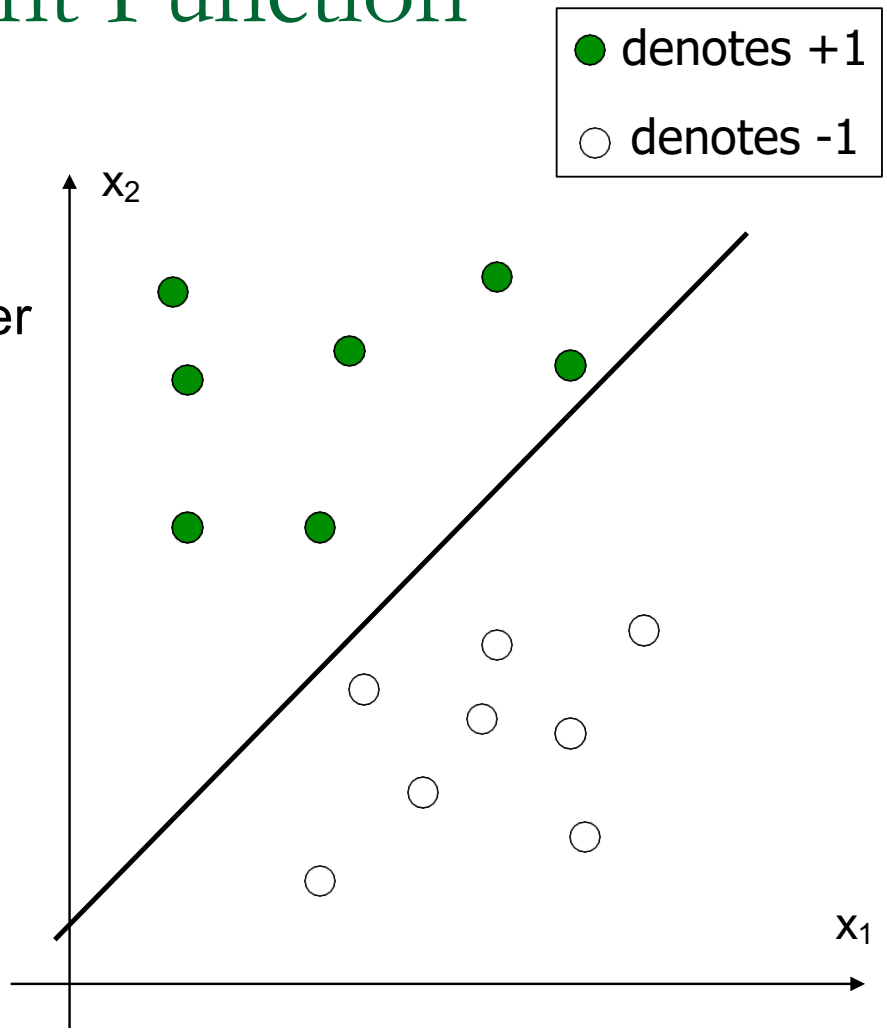
- Infinite number of answers!



Linear Discriminant Function

- How would you classify these points using a linear discriminant function in order to minimize the error rate?

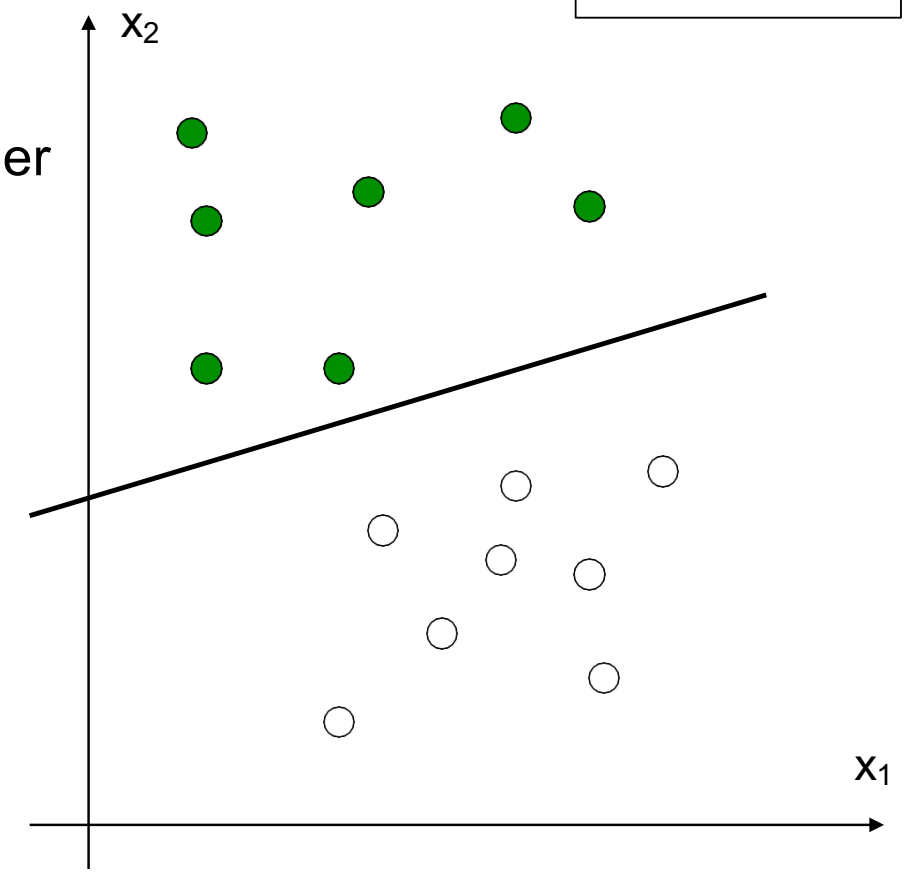
- Infinite number of answers!



Linear Discriminant Function

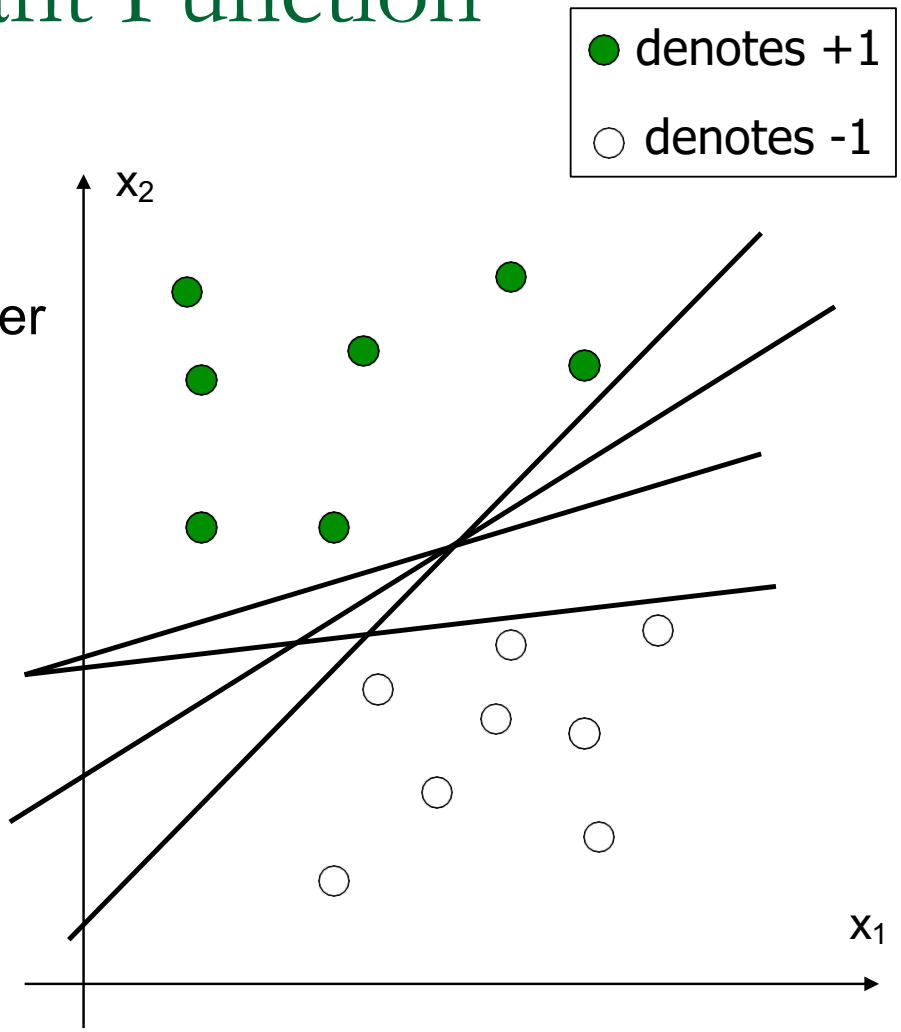
- How would you classify these points using a linear discriminant function in order to minimize the error rate?

- Infinite number of answers!



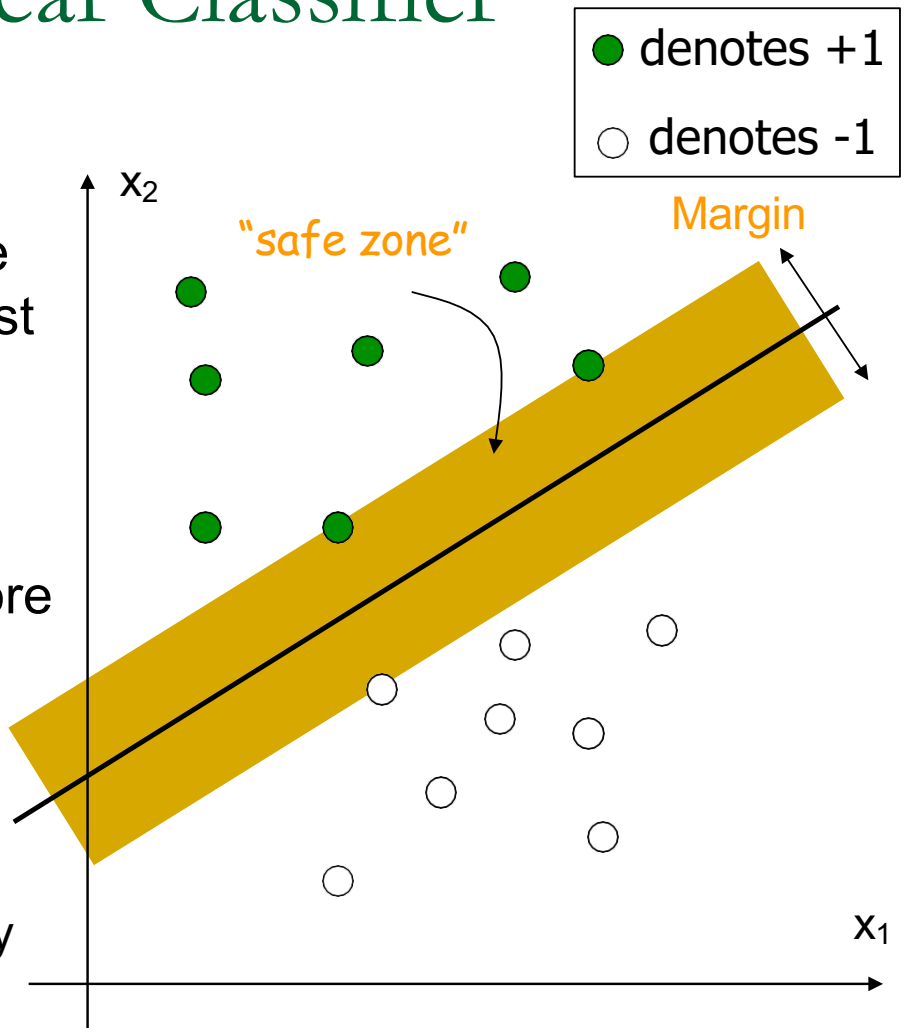
Linear Discriminant Function

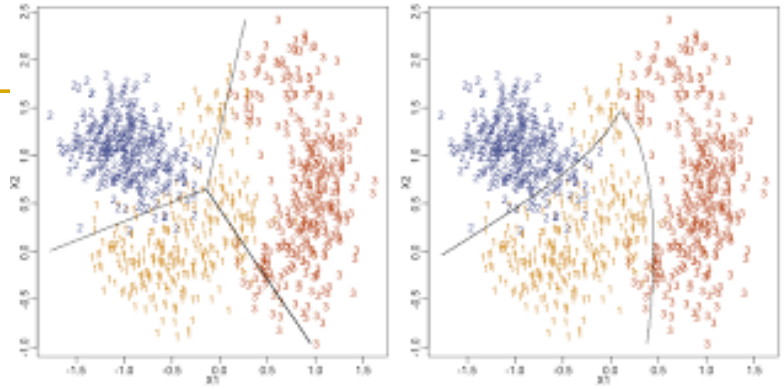
- How would you classify these points using a linear discriminant function in order to minimize the error rate?
- Infinite number of answers!
- Which one is the best?

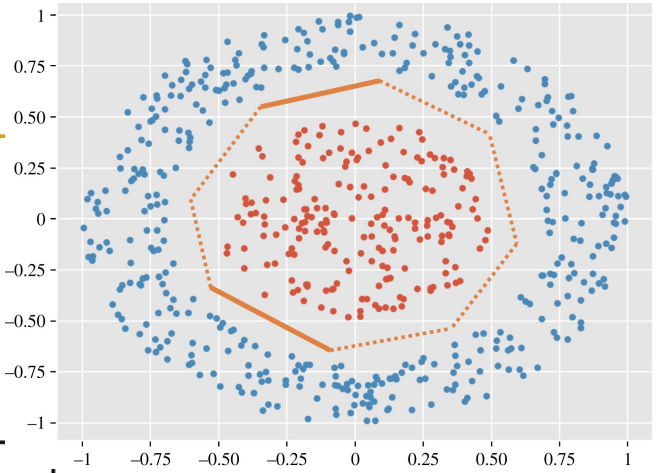


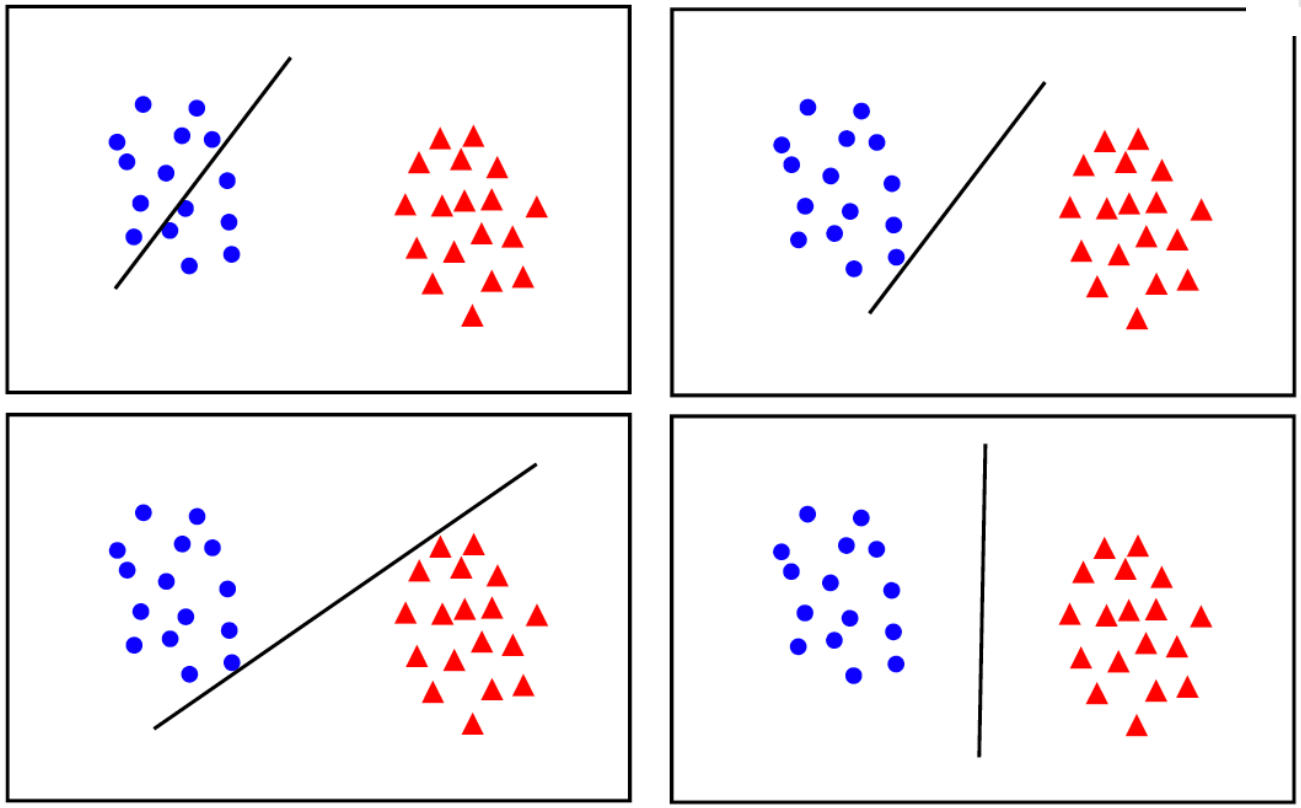
Large Margin Linear Classifier

- The linear discriminant function (classifier) with the maximum margin is the best
- Margin is defined as the width that the boundary could be increased by before hitting a data point
- Why it is the best?
  - Robust to outliers and thus strong generalization ability

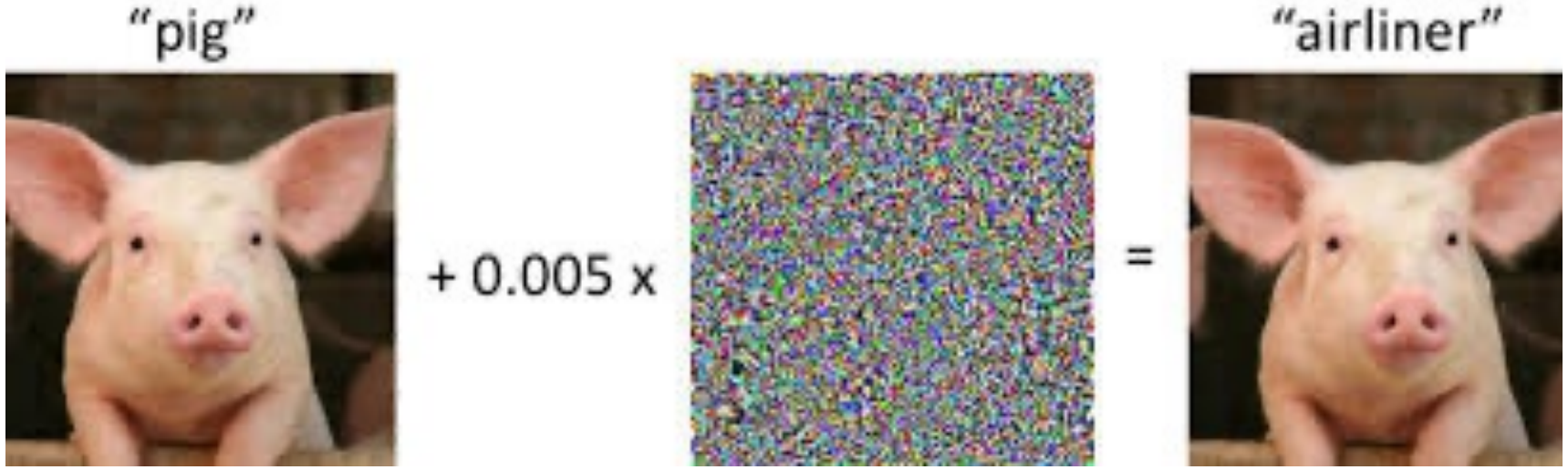


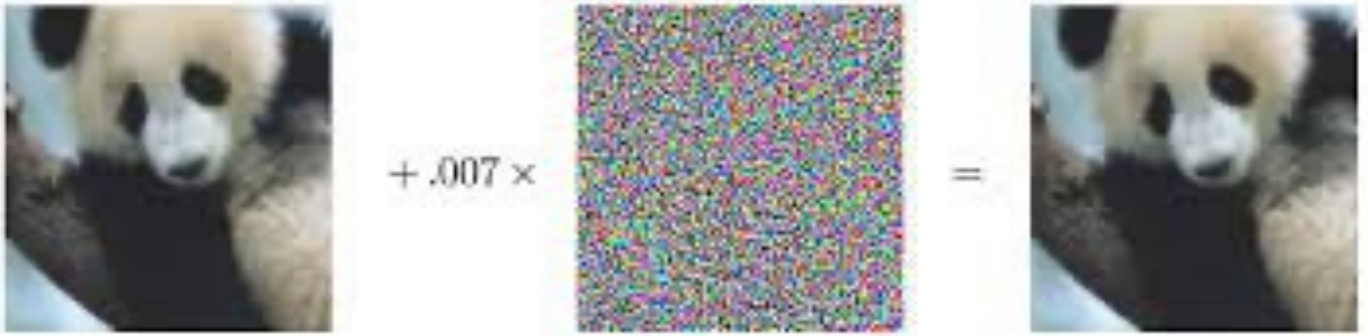






- maximum margin solution: most stable under perturbations of the inputs





x

"panda"
57.7% confidence

$\text{sign}(\nabla_x J(\theta, x, y))$

"nematode"
8.2% confidence

$x + \epsilon\text{sign}(\nabla_x J(\theta, x, y))$

"gibbon"
99.3 % confidence

Linear Discriminant Function

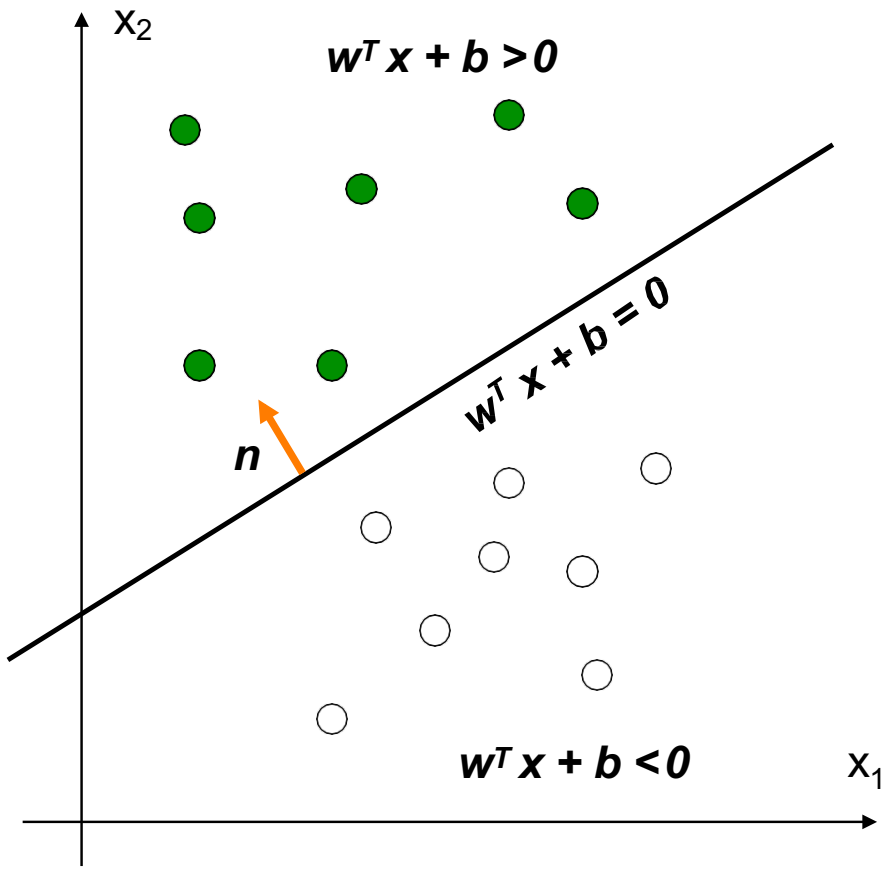
- f(x) is a linear function:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

- A hyper-plane in the feature space

- (Unit-length) normal vector of the hyper-plane:

$$\mathbf{n} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$$



Large Margin Linear Classifier

- Given a set of data points:

$\{(\mathbf{x}_i, y_i)\}, i = 1,2, \quad , n,$ where

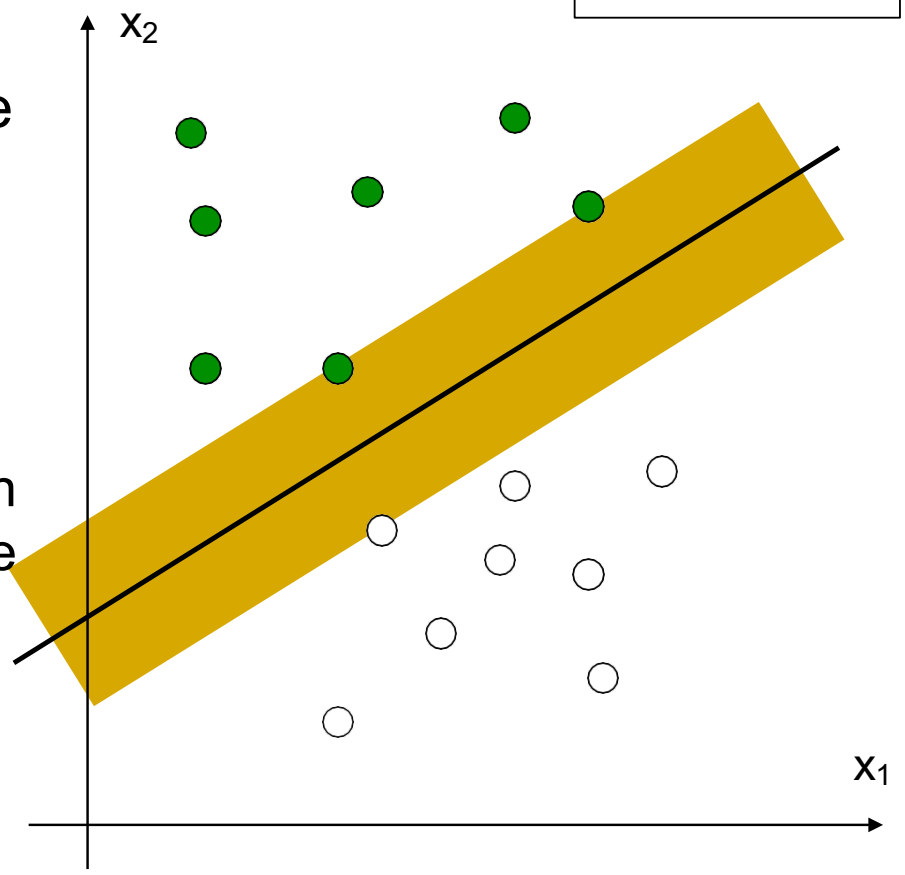
$$\text{For } y_i = +1, \ \mathbf{w}^T \mathbf{x}_i + b > 0$$

$$\text{For } y_i = -1, \ \mathbf{w}^T \mathbf{x}_i + b < 0$$

- With a scale transformation on both *w* and *b*, the above is equivalent to

$$\text{For } y_i = +1, \ \mathbf{w}^T \mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1, \ \mathbf{w}^T \mathbf{x}_i + b \leq -1$$



Linear Discriminant Function

- Since $\mathbf{w}^\top \mathbf{x} + b = 0$ and $c(\mathbf{w}^\top \mathbf{x} + b) = 0$ define the same plane, we have the freedom to choose the normalization of $\mathbf{w}$

- Choose normalization such that $\mathbf{w}^\top \mathbf{x}_+ + b = +1$ and $\mathbf{w}^\top \mathbf{x}_- + b = -1$ for the positive and negative support vectors respectively

- Then the margin is given by

$$\frac{\mathbf{w}}{||\mathbf{w}||} \cdot \left(\mathbf{x}_+ - \mathbf{x}_-\right) = \frac{\mathbf{w}^\top \left(\mathbf{x}_+ - \mathbf{x}_-\right)}{||\mathbf{w}||} = \frac{2}{||\mathbf{w}||}$$

Large Margin Linear Classifier

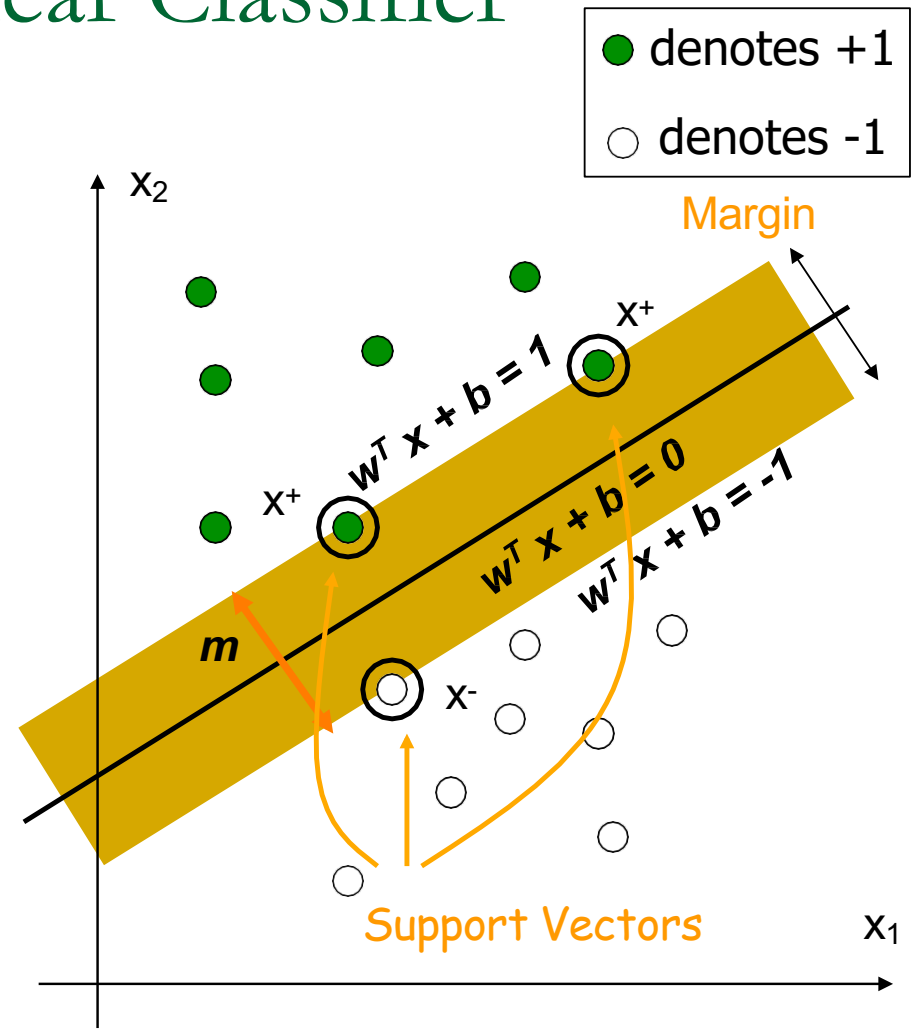
- We know that

$$\mathbf{w}^T \mathbf{x}^+ + b = 1$$

$$\mathbf{w}^T \mathbf{x}^- + b = -1$$

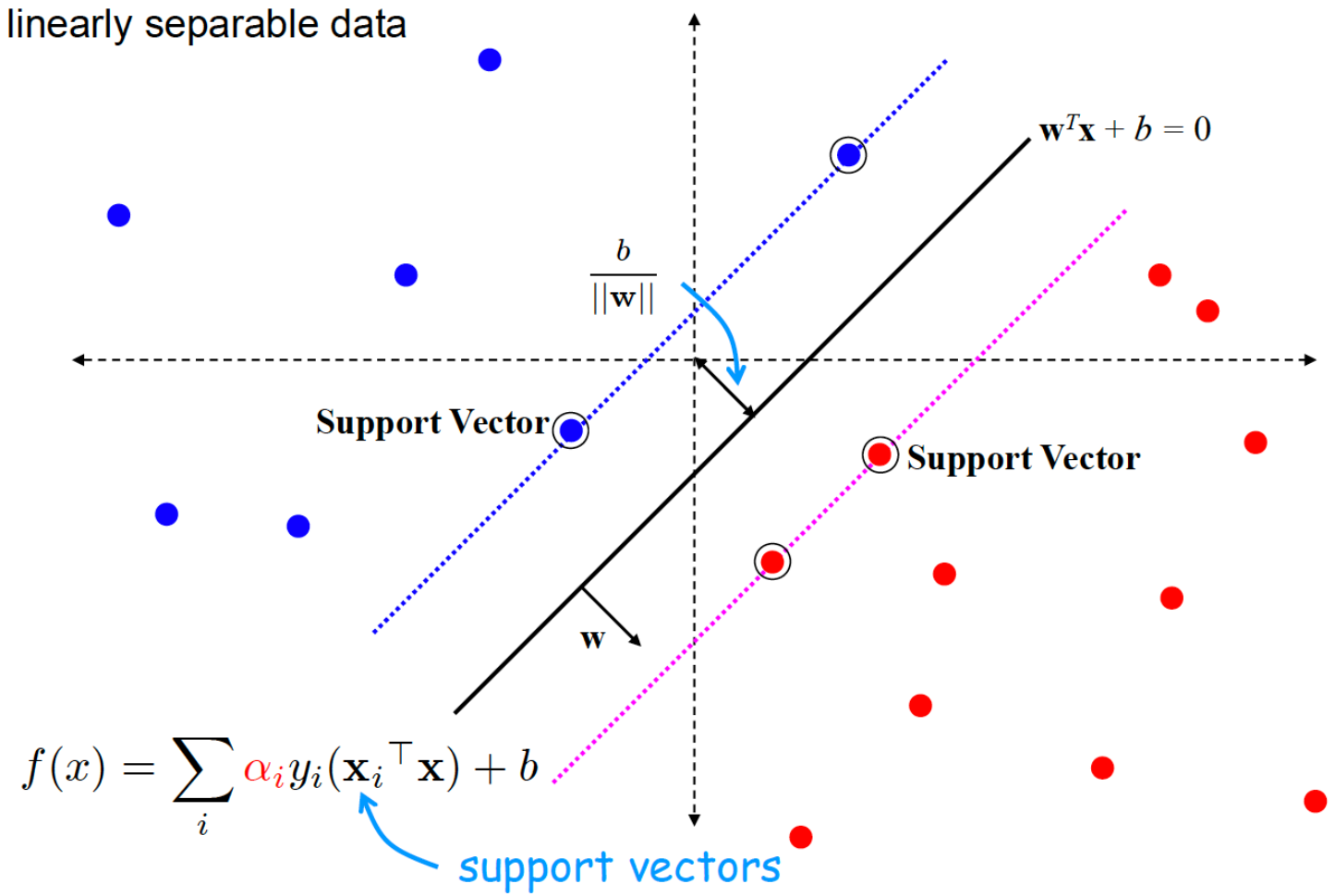
- Distance between the origin and the line $\mathbf{w}^t\mathbf{x}=k$ is $k/\|\mathbf{w}\|$
- The margin width is:

$$m = \frac{2}{||\mathbf{w}||}$$



$$\text{Width} = \frac{1-b}{|w|} - \frac{-b-1}{|w|} = \frac{2}{|w|}$$

Support Vector Machine



Large Margin Linear Classifier

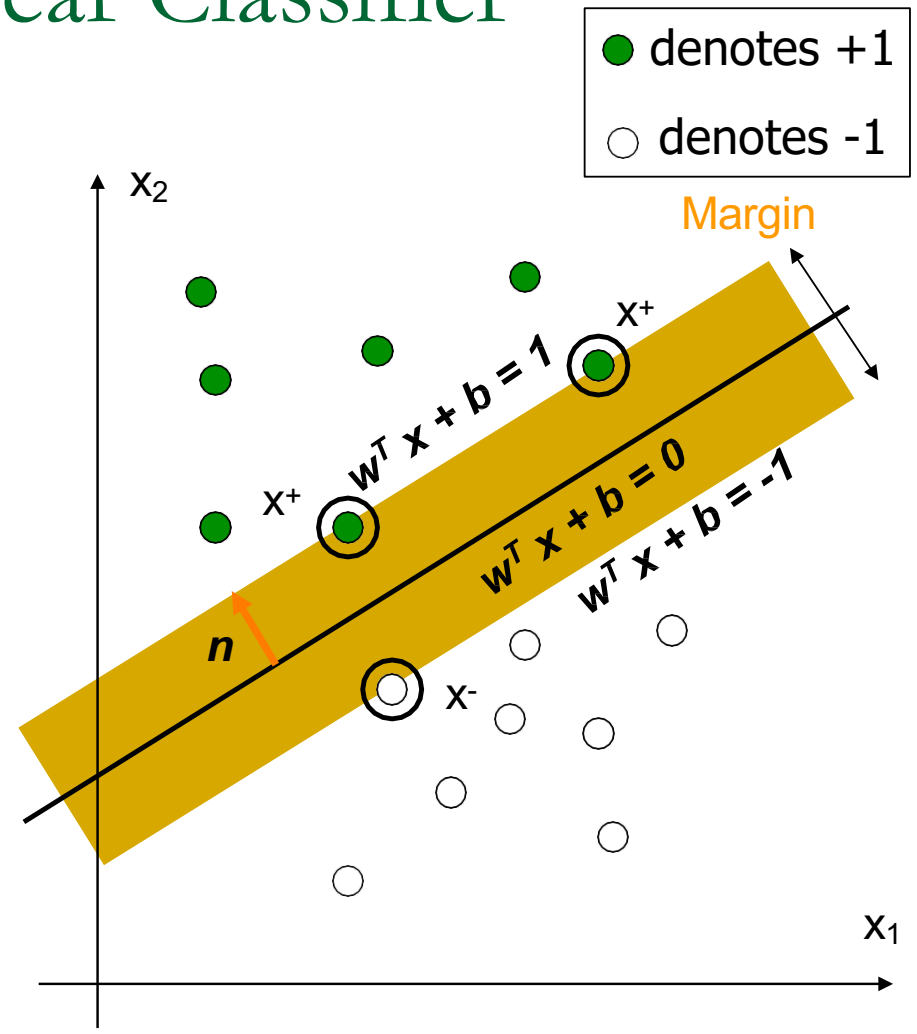
- Formulation:

$$\text{maximize} \quad \frac{2}{\|\mathbf{w}\|}$$

such that

$$\text{For } y_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1, \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1$$



Large Margin Linear Classifier

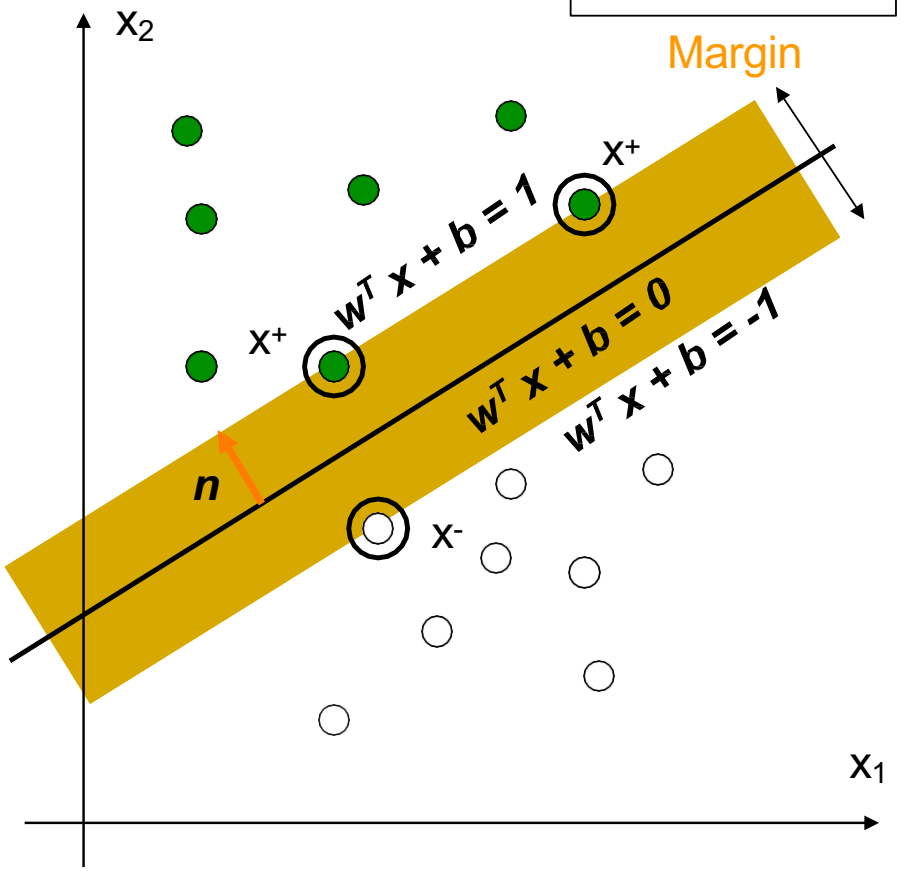
- Formulation:

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

such that

$$\text{For } y_i = +1, \quad \mathbf{w}^T\mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1, \quad \mathbf{w}^T\mathbf{x}_i + b \leq -1$$



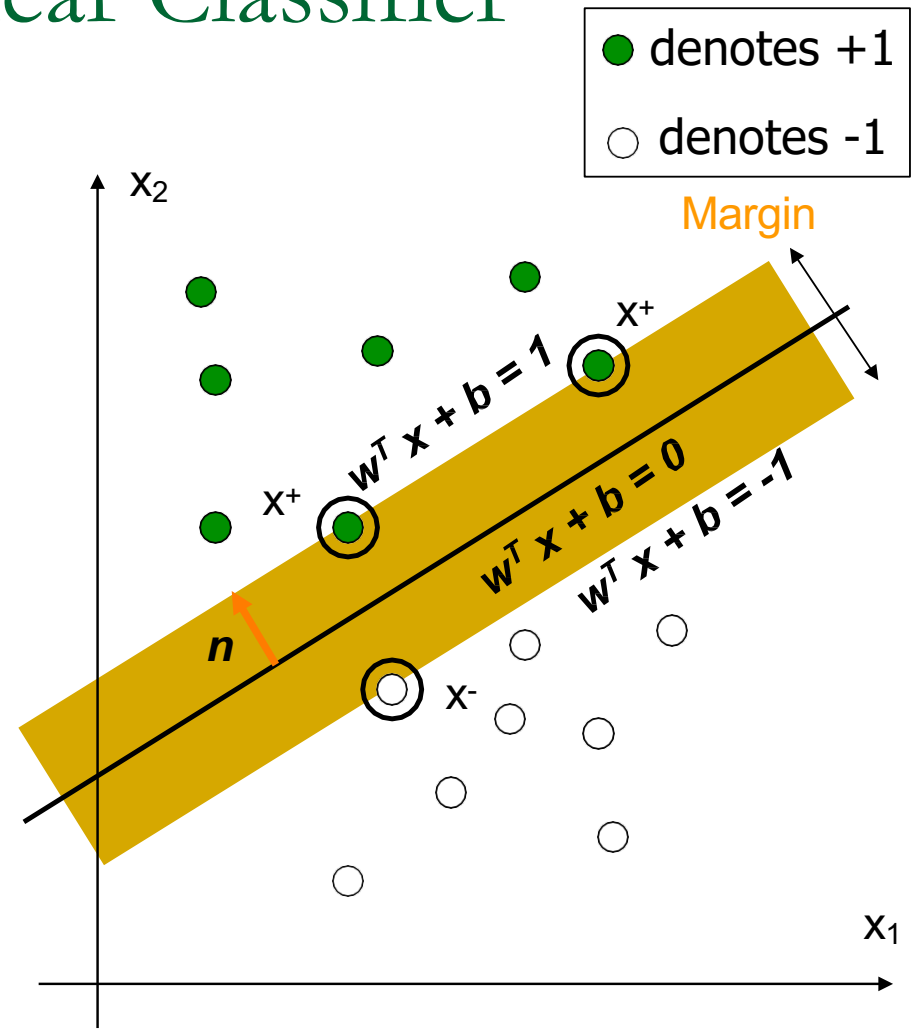
Large Margin Linear Classifier

- Formulation:

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

such that

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$



Solving the Optimization Problem

Quadratic programming with linear constraints

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{s.t.} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$

Solving the Optimization Problem

Quadratic programming with linear constraints

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{s.t.} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$



Lagrangian Function

$$\text{minimize} \quad L_p(\mathbf{w}, b, \alpha_i) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\right)$$

$$\text{s.t.} \quad \alpha_i \geq 0$$

Solving the Optimization Problem

$$\text{minimize} \quad L_p(\mathbf{w}, b, \alpha_i) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \right)$$

$$\text{s.t.} \qquad \alpha_i \geq 0$$

$$L = \frac{1}{2}|w|^2 - \sum_i \alpha_i \, y_{SVi} \, (\vec{w} \cdot \overrightarrow{x_{SVi}} + b) \; + \; \alpha_i$$

$$\frac{\partial L_p}{\partial \mathbf{w}} = 0 \quad \Rightarrow \quad \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial L_p}{\partial b} = 0 \quad \Rightarrow \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

$$L = \frac{1}{2}\left(\sum_i \alpha_i\, y_i\, \overrightarrow{x_{SV}i}\right)\left(\sum_j \alpha_j\, y_j\, \overrightarrow{x_{SV}j}\right)$$
$$- \sum_i \alpha_i\, y_i\, \overrightarrow{x_{SV}i} \cdot \left(\sum_j \alpha_j\, y_j\, \overrightarrow{x_{SV}j}\right) - \underbrace{\sum_i \alpha_i\, y_i\, b}_{\sum_i \alpha_i\, y_i = 0} + \sum_i \alpha_i$$

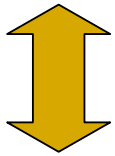
$$L = \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j\, \overrightarrow{x_{SV}i} \cdot \overrightarrow{x_{SV}j}$$

Solving the Optimization Problem

$$\text{minimize} \quad L_p(\mathbf{w}, b, \alpha_i) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \right)$$

$$\text{s.t.} \quad \alpha_i \geq 0$$

Primal formulation



Lagrangian Dual Problem

$$\textbf{Objective Function:} \quad \max_{\alpha} L_{dual} = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\textbf{Linear Constraints:} \quad \alpha_i \geq 0, \forall i \in D, \text{ and } \sum_{i=1}^{n} \alpha_i y_i = 0$$

Dual formulation

KKT Conditions

$$\frac{\partial}{\partial w_\nu} L_P = w_\nu - \sum_i \alpha_i y_i x_{i\nu} = 0 \quad \nu = 1, \cdots, d$$

$$\frac{\partial}{\partial b} L_P = -\sum_i \alpha_i y_i = 0$$

$$y_i(\mathbf{x}_i \cdot \mathbf{w} + b) - 1 \geq 0 \quad i = 1, \cdots, l$$

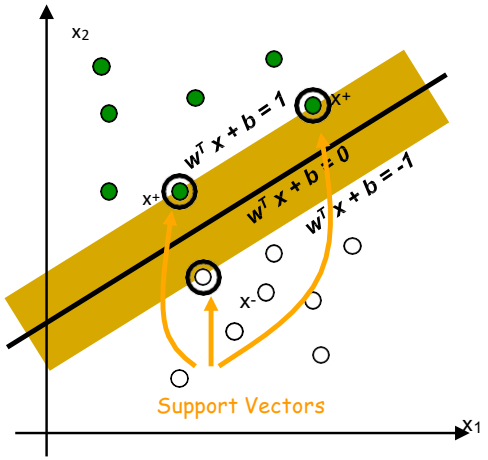
$$\alpha_i \geq 0 \quad \forall i$$

$$\alpha_i(y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1) = 0 \quad \forall i$$

Solving the Optimization Problem

- From KKT condition, we know:

$$\alpha_i \left(y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right) = 0$$



- Thus, only support vectors have $\alpha_i \neq 0$

- The solution has the form:

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = \sum_{i \in SV} \alpha_i y_i \mathbf{x}_i$$

get b from $y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0$,

where $\mathbf{x}_i$ is support vector

Solving the Optimization Problem

- The linear discriminant function is:

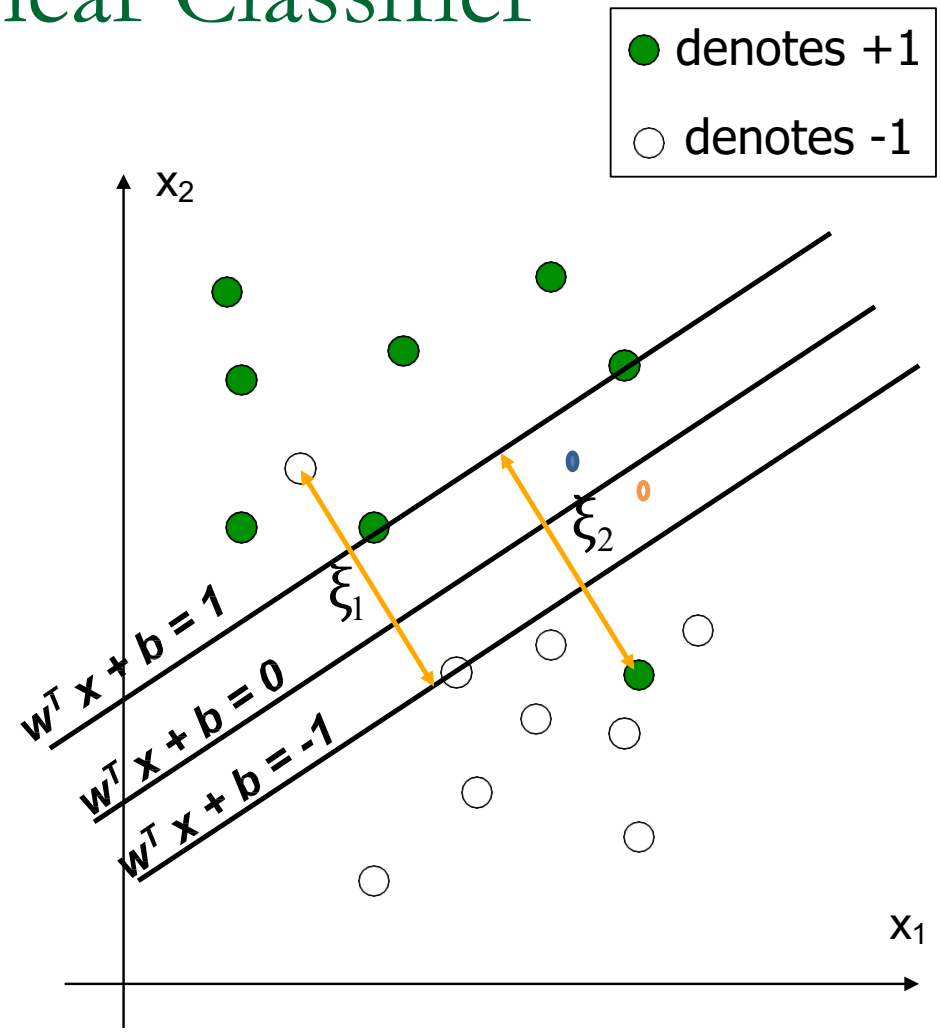
$$g(x) = w^T x + b = \sum_{i \in SV} \alpha_i y_i x_i^T x + b$$

- Notice it relies on a *dot product* between the test point $\boldsymbol{x}$ and the support vectors $\boldsymbol{x_i}$

- Also keep in mind that solving the optimization problem involved computing the dot products $\boldsymbol{x_i^T x_j}$ between all pairs of training points

Large Margin Linear Classifier

- What if data is not linear separable? (noisy data, outliers, etc.)
- Slack variables ξ_i can be added to allow mis-classification of difficult or noisy data points



Large Margin Linear Classifier

- Formulation:

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

such that

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

- Parameter *C* can be viewed as a way to control over-fitting.

Large Margin Linear Classifier

- Formulation: (Lagrangian Dual Problem)

$$\text{maximize} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

such that

$$0 \leq \alpha_i \leq C$$

$$\sum_{i=1}^{n} \alpha_i y_i = 0$$

A small value for C will increase the number of training errors, while a large C will lead to a behavior similar to that of a hard-margin SVM

- The parameter C controls the trade off between errors of the SVM on training data and margin maximization ($C = \infty$ leads to hard margin SVM).

- If it is too large, we have a high penalty for nonseparable points and we may store many support vectors and overfit. If it is too small, we may have underfitting.

Multiclass Classification

Input: hand-written character; Output: which character?

Input: a photograph of an object; Output: which of a set of categories of objects is it?

- Eg: the Caltech 256 dataset




Car tire


Car tire


Duck


laptop

OVA Multiclass SVM

Assumption: Each class individually separable from ***all*** the others

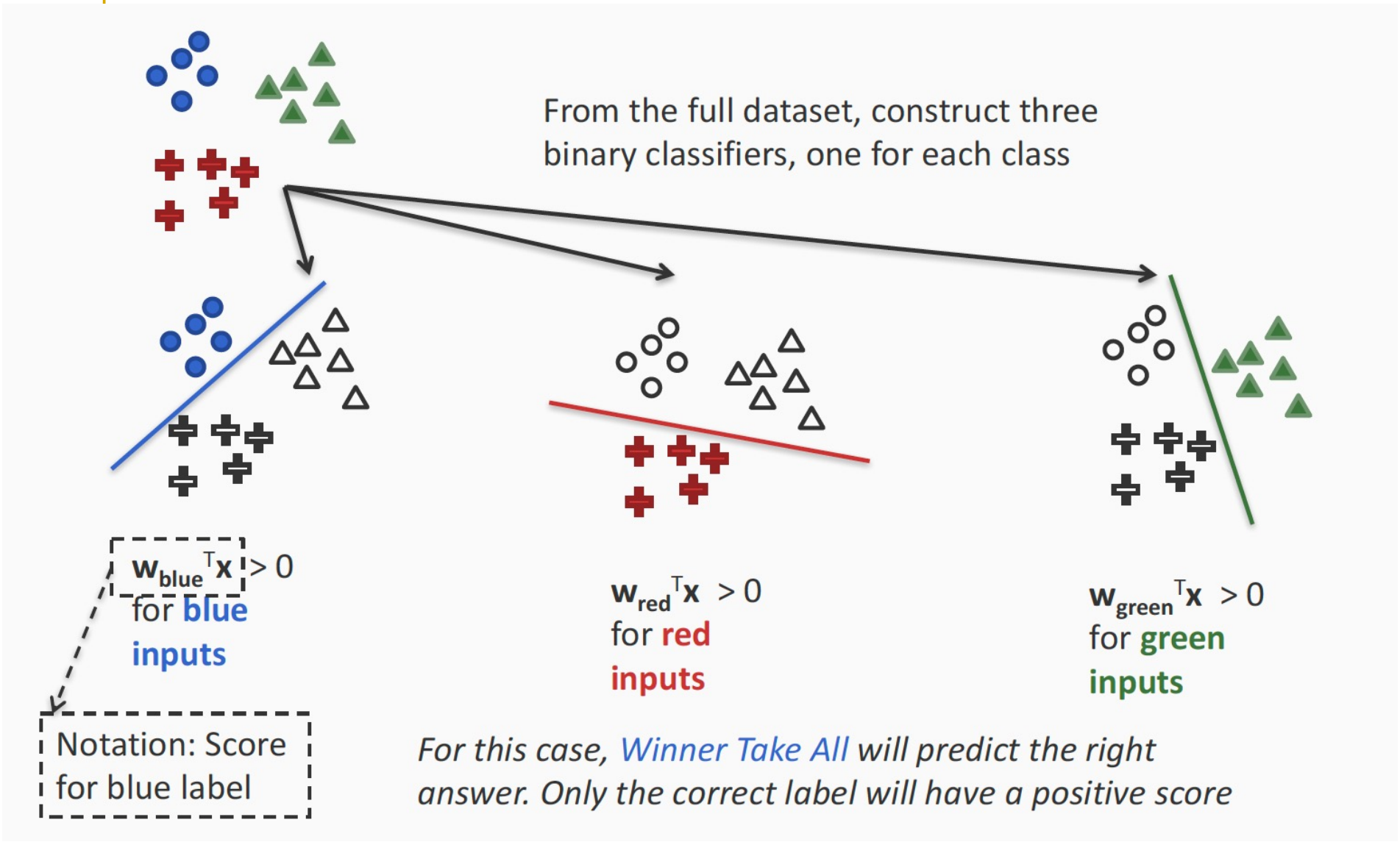
- Learning: Given a dataset $D = \{\langle \mathbf{x}_i, \mathbf{y}_i \rangle\}$,

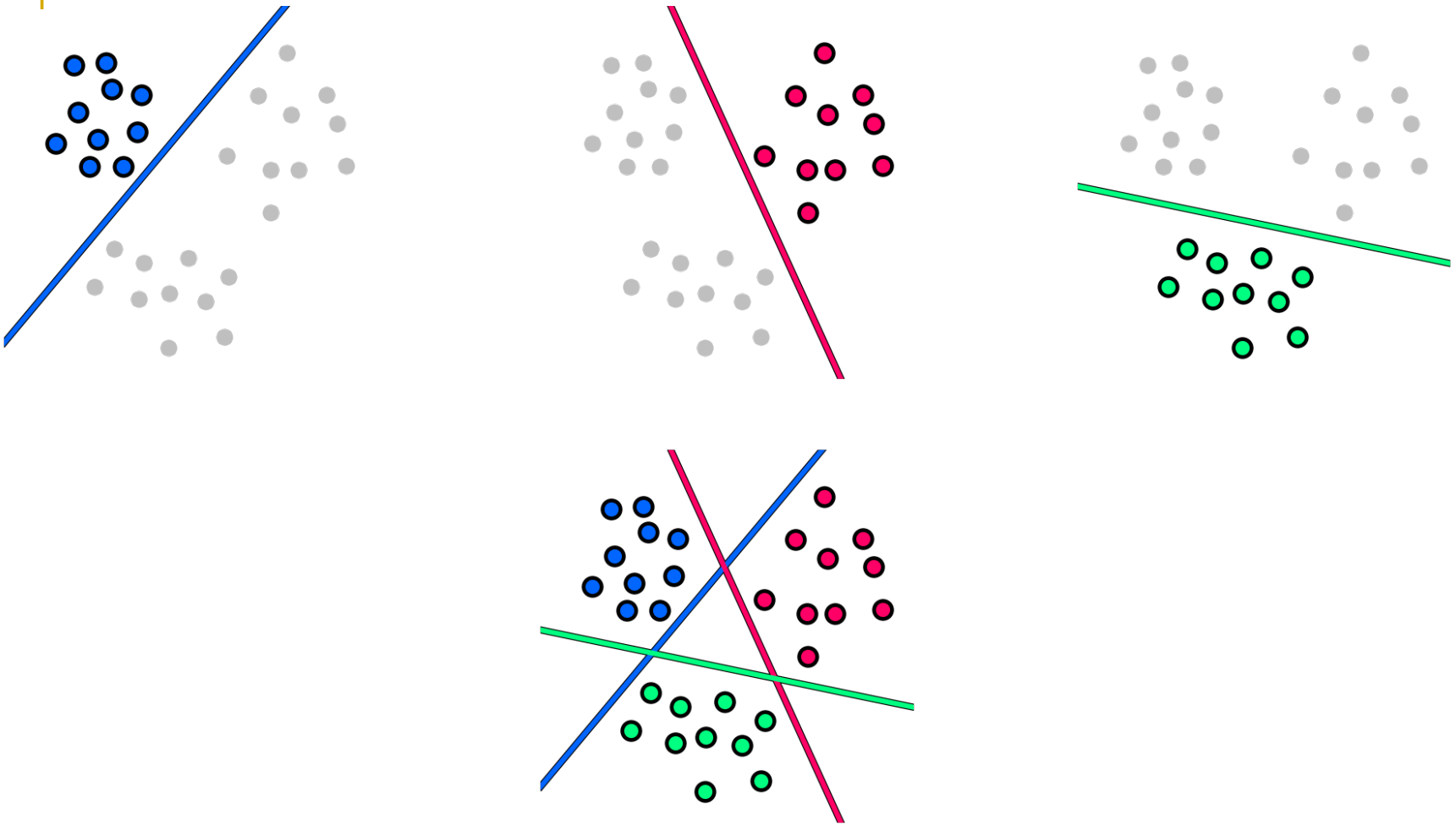
  Note: $\mathbf{x}_i \in \Re^n$, $\mathbf{y}_i \in \{1, 2, \cdots, K\}$

  – Decompose into K binary classification tasks
  – For class k, construct a binary classification task as:
    - Positive examples: Elements of D with label k
    - Negative examples: All other elements of D
  – Train K binary classifiers $\mathbf{w}_1, \mathbf{w}_2, \cdots \mathbf{w}_K$ using any learning algorithm we have seen

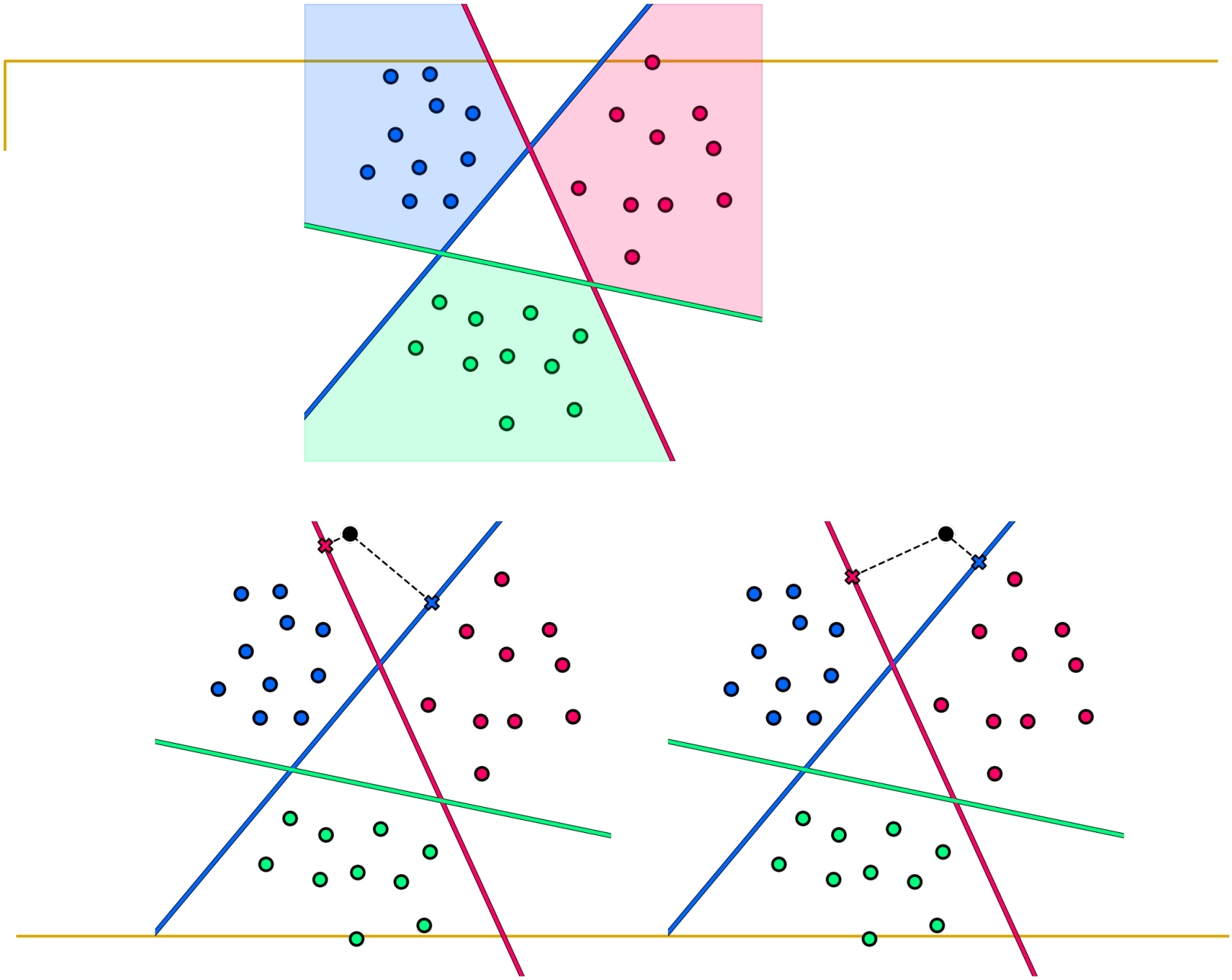
- Prediction: "*Winner Takes All*"

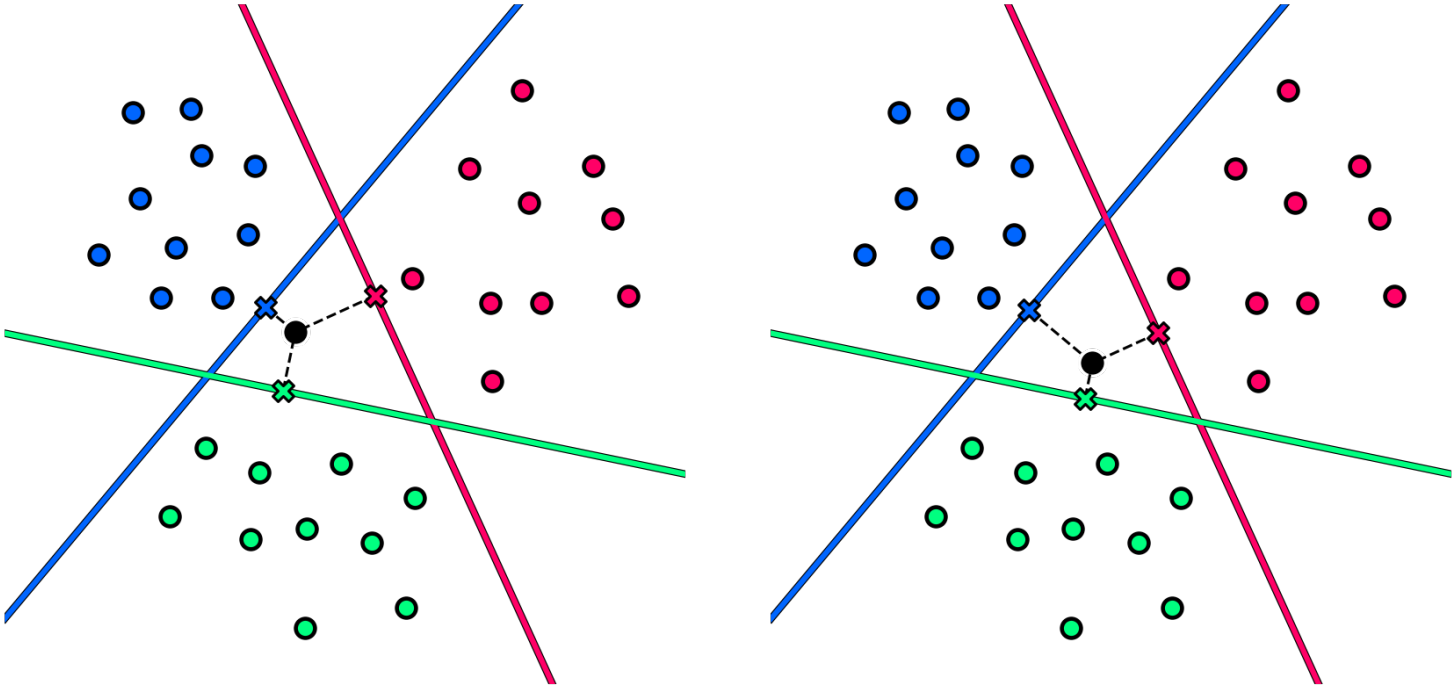
  $\text{argmax}_i \mathbf{w}_i^T \mathbf{x}$

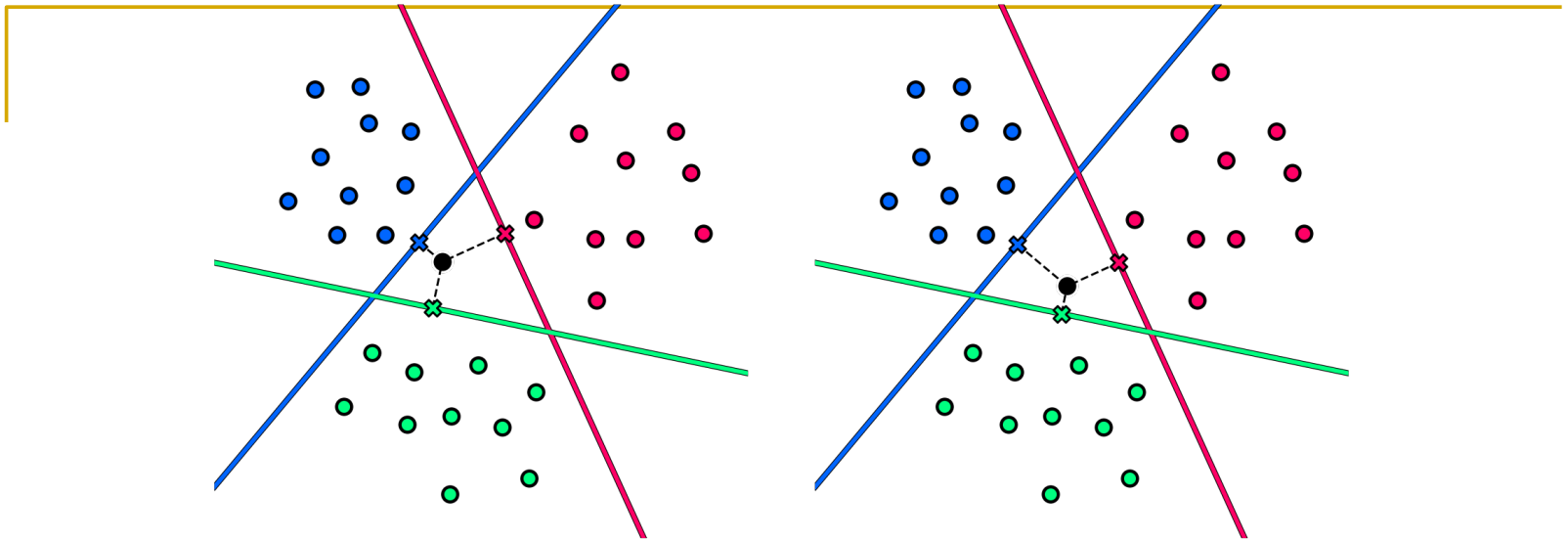

From the full dataset, construct three binary classifiers, one for each class
$w_{blue}^T x > 0$ for blue inputs
$w_{red}^T x > 0$ for red inputs
$w_{green}^T x > 0$ for green inputs
Notation: Score for blue label
For this case, Winner Take All will predict the right answer. Only the correct label will have a positive score

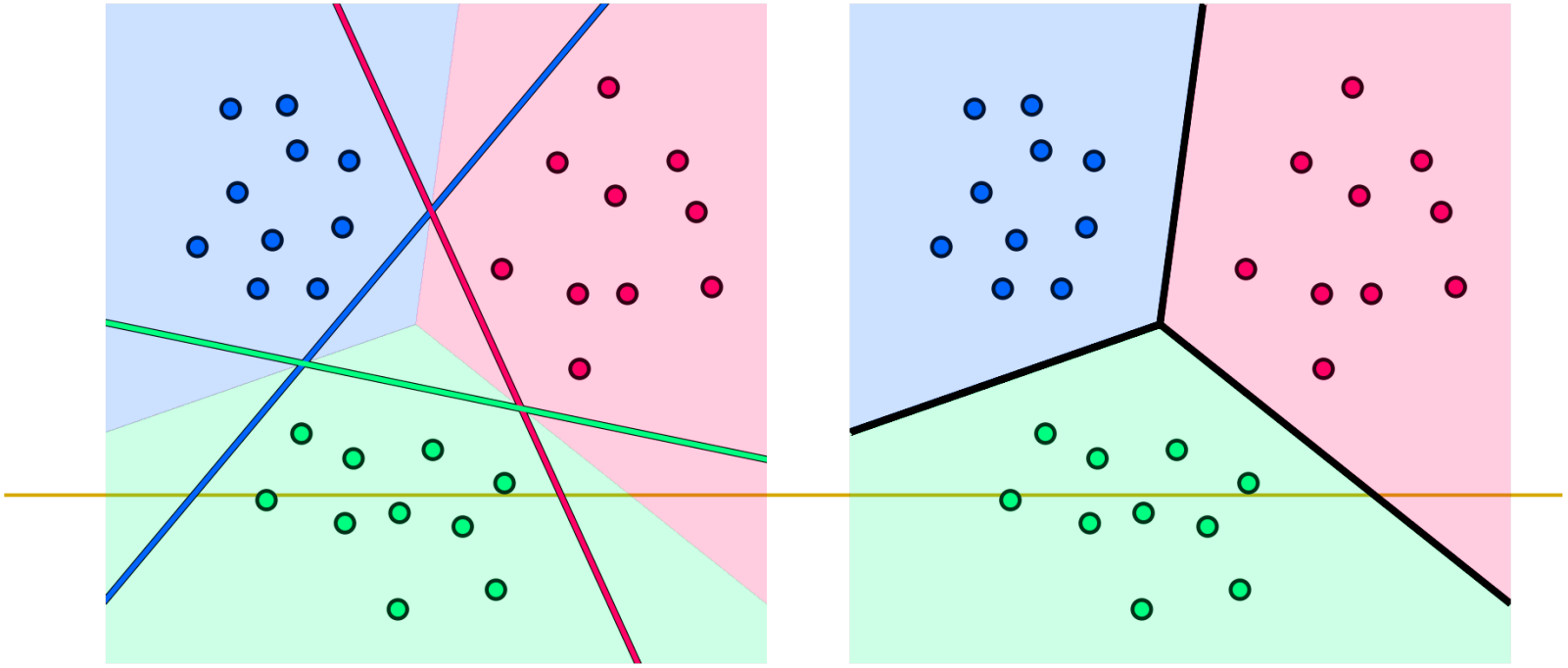


https://jermwatt.github.io/machine_learning_refined/notes/7_Linear_multiclass_classification/7_2_OvA.html









What are the challenges with OVA?

- Multiple times training
- Computationally expensive
- Class imbalance

OVO Classification

Assumption: *Every* pair of classes is separable

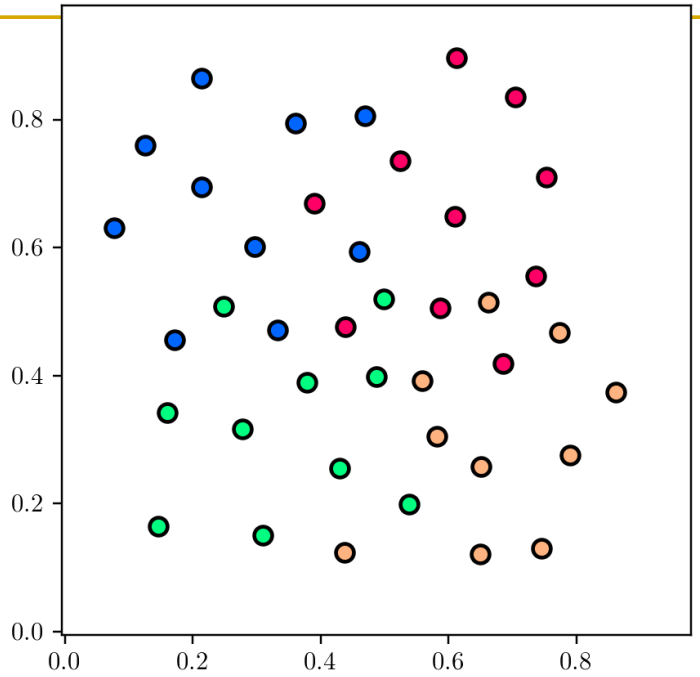
- Learning: Given a dataset D = $\{\langle \mathbf{x}_i, \mathbf{y}_i \rangle\}$,

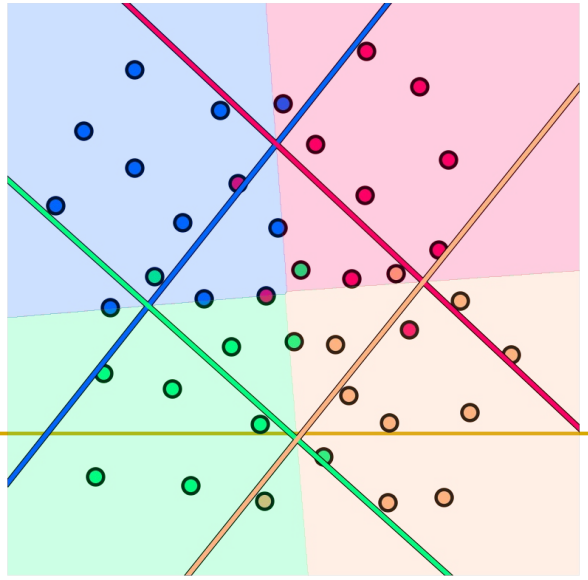
  Note: $\mathbf{x}_i \in \Re^n$, $\mathbf{y}_i \in \{1, 2, \cdots, K\}$

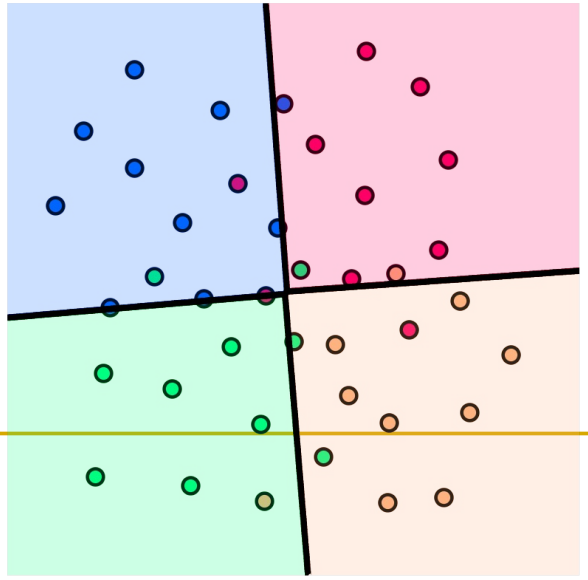
  – For every pair of labels (j, k), create a binary classifier with:
    - Positive examples: All examples with label j
    - Negative examples: All examples with label k

  – Train $\binom{K}{2} = \frac{K(K-1)}{2}$ classifiers in all

- Prediction: More complex, each label get K-1 votes
  – How to combine the votes? Many methods
    - Majority: Pick the label with maximum votes
    - Organize a tournament between the labels


0.0
0.2
0.4
0.6
0.8
0.0
0.2
0.4
0.6
0.8





OVA vs OVO

Viewed naively, AVA seems faster and more memory efficient. It requires O(N2) classifiers instead of O(N), but each classifier is (on average) much smaller. If the time to build a classifier is superlinear in the number of data points, AVA is a better choice. With SVMs, AVA's probably best.